

Project 7 (Bonus) — MCPForge-Local-Notes-And-Tasks-MCP-Server

Type: MCP Server (2 Tools, 1 Resource, 1 Prompt) · **Stack:** Python, official `mcp` SDK (FastMCP), SQLite, MCP Inspector · **Level:** Intermediate (bonus / portfolio project) **Tagline:** A real MCP server — Local Notes & Tasks — exposing 2 Tools, 1 Resource, and 1 Prompt over the Model Context Protocol, usable by any MCP-compatible client without a custom integration for each one.

Not part of the numbered Doc01-19 sequence in the way Projects 1-6 are — this bonus project teaches one specific technology, MCP server-building, using skills you already have from [01_python_foundations](#) (config/logging) and [06_tools_function_calling](#) (precise tool descriptions, and Doc06's own MCP sections). Full detail lives here, and only here.

Charter (what this project is)

A real MCP (Model Context Protocol) server, built with the official `mcp` Python SDK, that exposes 2 Tools, 1 Resource, and 1 Prompt over stdio — something any MCP-compatible client (Claude Desktop, your own agent code, an IDE) can connect to and use, without you writing a custom integration for each one. This project proves two things with real code, not just reading: (1) you understand the actual mechanics of an MCP server — Tools, Resources, Prompts, and the client/server handshake underneath them — not just the concept, and (2) you can tell, for a real piece of data or behavior, which of the three it actually belongs on.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Imagine you've built several small AI features across Docs 01-06 and Projects 1-6 — a chat agent here, a tool-calling agent there. Each one has its own hardcoded tools, wired directly into that one app's API call. Now imagine you want the same "notes and tasks" capability available not just in one app, but in Claude Desktop, in your own agent code, and in an IDE plugin — three completely different programs, none of which know anything about each other or about your specific database.

Without MCP, you'd write three separate integrations — the same add-a-note logic, copy-pasted and adapted three times, drifting out of sync the moment one copy changes. MCPForge is the alternative: you build the notes/tasks logic exactly once, as an MCP **server**, and expose it over the protocol. Any MCP-compatible **client** — Claude Desktop, a script you write, an IDE — can connect

to that one server and discover what it offers, at runtime, without you writing a single line of client-specific integration code.

The server you're building, MCPForge, is a small but real "Local Notes & Tasks" tool: `add_note` and `search_notes` as Tools (they *do* something — write or search), `notes://all` as a Resource (it's read-only data, nothing to act on), and `summarize_notes` as a Prompt (a reusable template, not a hardcoded string buried in one app). Getting each of these three right — not just cramming everything into Tools because that's the one you understand best — is the actual point of this project.

Each Step below builds one piece, in order: Step 1 proves the connection itself works, with the smallest possible server. Step 2 makes the data real — a SQLite database that survives a restart, with input that's validated instead of silently accepted. Step 3 adds the Resource and the Prompt, and forces you to think about *why* each one isn't just a third and fourth Tool. Step 4 builds the client side — a script that connects, discovers, and uses everything the server offers — proving the full loop actually works end to end.

Before you read further — think about it yourself: `add_note` clearly needs to be a Tool — it changes something. But `notes://all`, a plain listing of every note, *could* technically be built as a third Tool instead of a Resource. What would you actually lose by doing that? And `summarize_notes` could just be a prompt string you type into Claude Desktop by hand, instead of a Prompt the server provides — what does having the server provide it actually buy you, that typing it yourself doesn't? Sit with these for a minute before you read the Steps below.

Where This Fits

Projects 1-6 in this curriculum are all one continuous arc — the same core skills (LLM calls, tools, LangGraph, multi-agent design, production wrapping), applied to a growing number of agents, then proven on a second problem domain in Project 6. This project is not part of that arc. It doesn't add agents, and it isn't built on LangChain or LangGraph at all — there's no LLM call anywhere inside the server you're building here.

What this project teaches instead is one specific, widely-used technology: **MCP server-building**. Docs 01-06 already taught you the underlying ideas MCP standardizes — a tool's precise description, its typed argument shape, clean error handling instead of a crash (all from Doc06), and config/logging discipline (Doc01). MCP itself, covered in Doc06's later sections, is the protocol that lets you expose those same ideas *once*, over a standard connection, instead of re-wiring them by hand into every app that wants to use them.

Build this project because MCP is genuinely used in real tools right now — Claude Desktop, Cursor, and other IDEs all speak it — and "I built and tested a real MCP server" is a specific, checkable

portfolio line that a generic "I called an LLM API" project can't match. It also previews being a capable MCP *client*, not just a server author — Step 4 here is a small, deliberate first taste of that, done properly in a later bonus project.

Setup (do this once, before Step 1)

These are the very first commands to run. You don't need to read anything else first.

```
cd project_7_mcp_server
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install "mcp[cli]" python-dotenv pydantic
pip freeze > requirements.txt
```

Create a file called `.env` in this folder (never commit this file to git):

```
LOG_LEVEL=INFO
NOTES_DB_PATH=notes.db
```

Create a second file, `.env.example`, with the same keys:

```
LOG_LEVEL=INFO
NOTES_DB_PATH=notes.db
```

Neither key here is a secret, so real-looking values are fine in both files — this project never touches an LLM API key at all, since the server itself doesn't call a model.

Add `.venv/` and `notes.db` to your `.gitignore` before your first commit — the database file is generated locally by Step 2, not something to commit.

`mcp[cli]` installs the **MCP Inspector**, a small local web UI that connects to your server, lists its Tools/Resources/Prompts, and lets you call them by hand — the fastest way to test a server before writing any client code of your own. Run it against your server with `mcp dev server.py`.

A Real Example (so this isn't just theory)

The Tool/Resource/Prompt part of this project needs a real example to work with — not just "store some notes." Use this one, or make your own, but keep it this specific:

Scenario: a small local notes tool. Each note is just free text plus when it was created.

Test these 3 inputs for `add_note`: 1. "Buy milk and eggs" → succeeds, returns something like "Note added with id 1." 2. "Call the dentist to reschedule" → succeeds, returns "Note added with id 2." 3. "" (empty string) or " " (only whitespace) → must be **rejected** with a clear error, not silently stored as a blank note.

Then: - `search_notes("dentist")` → returns ["Call the dentist to reschedule"] — note 1 doesn't match, and note 3 was never stored at all. - Reading the `notes://all` Resource → returns a read-only listing of both real notes, with their ids. - Using the `summarize_notes` Prompt with a date range covering today → returns a filled-in prompt *template* asking a model to summarize the notes created in that range. The server produces the prompt text, not a summary — whichever client you're using is the one that actually sends that prompt to a model.

Why input 3 matters: it's the same "don't let bad input slide through" lesson Project 1's `customer_name=None` case taught for a missing field — here it's an empty *required* field instead, and the fix takes the same instinct: catch it on purpose, fail with a clear message, and never store something you already know is invalid.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** every AI app that wants your notes/tasks logic currently needs its own custom integration, written by hand. This project builds that logic exactly once, behind a standard protocol, so any MCP-compatible client can use it without a rewrite.
- **Why it matters:** MCP is a real, currently-used standard — Claude Desktop, Cursor, and other developer tools all speak it. "I built and tested a real MCP server, not just a toy script" is a specific, checkable claim an interviewer can verify by actually running your code.
- **When you'd build something like this at a real job:** any internal tool, data source, or workflow you want multiple AI apps or agents to share, without maintaining a separate hand-written integration for every one of them.
- **How it's built:** a `FastMCP` server exposing 2 Tools (an action with a side effect — adding a note; a read query — searching notes), 1 Resource (a read-only listing, no action attached), and 1 Prompt (a reusable, parameterized template) — all backed by a real SQLite database, not an in-memory list that forgets everything the moment the process restarts.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The MCP Inspector connects, but <code>add_note</code> doesn't show up in the tool list	Check the function is actually decorated with <code>@mcp.tool()</code> , and that <code>mcp.run(transport="stdio")</code> is the last line that actually runs — a server that never calls <code>mcp.run()</code> never starts listening
<code>add_note("")</code> silently creates a blank note	Validate <code>text.strip()</code> before touching the database, and raise a clear error if it's empty — Doc01's "fail loudly, not

Problem	Fix
	silently" idea, applied here to a tool's own input content, not just to missing config
Notes disappear every time you restart the server	You're still using Step 1's in-memory list. Step 2 replaces it with a real SQLite file (<code>sqlite3.connect(db_path)</code>), so notes survive a restart
<code>search_notes("Dentist")</code> finds nothing, even though a note says "dentist"	Your search is doing an exact, case-sensitive match. Use SQL <code>LIKE</code> with <code>%query%</code> , and lowercase both sides before comparing
The client hangs forever and never gets a reply	Something inside the server is writing to stdout directly (a stray <code>print()</code>) — on the stdio transport, stdout <i>is</i> the protocol channel. Route all logging to stderr (or a file) through <code>logging_setup.py</code> , never <code>print()</code>
You can't decide whether something should be a Tool or a Resource	Ask one question: does calling it <i>change</i> anything, or cost real computation? If yes, it's a Tool. If it's only handing back data to read, it's a Resource

Built During These Documents

[01_python_foundations](#) → [06_tools_function_calling](#) (including its MCP sections)

Plan Before You Code

Same process as every project (see [15_five_projects_index](#)): Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks. Write it down yourself before you open your editor — even for a project this size, deciding up front which of the 4 things (2 Tools, 1 Resource, 1 Prompt) each new piece of behavior belongs to is exactly the kind of design decision this process is for.

How To Build This — Step by Step

Step 1 — A Minimal Server With One Tool

Project: MCPForge-Local-Notes-And-Tasks-MCP-Server — Step 1 of 4: A Minimal Server With One Tool

What this step does: proves the most basic thing works — your code can start an MCP server, and a client (the Inspector, or a tiny script) can connect to it and successfully call its one tool. Nothing else yet. **Why this step matters:** every layer you add after this (real persistence, Resources,

Prompts) is worthless if the basic protocol handshake doesn't work — this step isolates that handshake from everything else, so if something breaks later, you already know the connection itself isn't the cause. **When you'll hit this for real:** the first thing you should do on *any* new MCP server, professional or personal — before adding real logic, prove a client can actually connect and call the simplest possible tool. **Read first:** [01_python_foundations Core Concepts](#) (config + logging), [06_tools_function_calling Core Concepts](#) — "What MCP actually is, and the problem it solves" and "MCP servers and clients, and how they actually connect."

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build `config.py` and `logging_setup.py` (or reuse them from Doc01 if you already built them there). `LOG_LEVEL` is the only setting you need this step. 2. Install the official `mcp` SDK (`pip install "mcp[cli]"`) and write `server.py`: a `FastMCP` instance, with one tool, `add_note(text: str) -> str`, that appends to a plain in-memory Python list (no database yet) and returns a confirmation string. 3. Run the server with the MCP Inspector (`mcp dev server.py`) and call `add_note` by hand through its UI — confirm it responds correctly. If you'd rather not use the Inspector, write a short client script using `stdio_client / ClientSession` (the shape is in Doc06's MCP section) that calls `add_note` once and prints the result. 4. Add a note twice in the same session and confirm both calls succeed — this in-memory list is expected to forget everything the moment the server process stops. That's the exact problem Step 2 fixes.

Your files after Step 1:

```
project_7_mcp_server/
├── .env / .env.example    → LOG_LEVEL (reused pattern from Doc01)
├── config.py              (reused from Doc01)
├── logging_setup.py       (reused from Doc01)
└── server.py              → FastMCP("notes-server"); one tool: add_note(text
```

Step 2 — A Real SQLite-Backed Store, and the Search Tool

Project: MCPForge-Local-Notes-And-Tasks-MCP-Server — Step 2 of 4: A Real SQLite-Backed Store, and the Search Tool

What this step does: replaces the in-memory list with a real SQLite database, so notes survive a server restart, and adds the second Tool, `search_notes`. **What's new vs. Step 1:** a `db.py` module with real persistence, an `exceptions.py` with a custom error for invalid input, and input validation on `add_note` that rejects empty/whitespace-only text instead of silently storing it. **What stays the same:** the server still only exposes Tools — Step 3 is where the Resource and the Prompt show up. **Why this step matters:** an MCP server that forgets everything on restart isn't something a real client would actually rely on — and a tool that silently accepts garbage input (an empty note) is exactly the kind of bug Doc01's "fail loudly, not silently" idea, and Doc06's Pydantic-argument-checking idea, both exist to prevent, just applied one level deeper than the argument's *shape* — this time it's the argument's *content* that needs checking. **When you'll hit this for real:** any tool backed

by real storage — a note, a task, a database row — needs this exact pair of fixes: real persistence, and validation that rejects bad input before it's ever written. **Read first:** [06_tools_function_calling Core Concepts — "Checking arguments: Pydantic as the contract"](#) and "Getting a tool's failure back to the model, correctly."

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Add `NOTES_DB_PATH` to your config. Write `db.py : init_db(path)` (creates the `notes` table if it doesn't exist), `insert_note(path, text)`, and `find_notes(path, query)` using SQL `LIKE` for a real substring search. 2. Write `exceptions.py` → `EmptyNoteError(Exception)`. In `server.py`'s `add_note`, check `text.strip()` before calling `insert_note()` — raise `EmptyNoteError` with a clear message if it's empty, and let the tool's `except` block turn that into a clean error string the client sees (the same pattern as Doc06's tool-failure-handling section), not a crash. 3. Add the second tool, `search_notes(query: str) -> list[str]`, calling `find_notes()`. 4. Test: add 2-3 real notes, restart the server process entirely, and confirm `search_notes` still finds them — this is the actual proof persistence works, not just that the code runs once. Then call `add_note("")` and `add_note(" ")` and confirm both are rejected with a clear message, not silently stored.

Your files after Step 2:

```
project_7_mcp_server/
├─ .env / .env.example
├─ config.py           → now also loads NOTES_DB_PATH
├─ logging_setup.py
├─ exceptions.py       → EmptyNoteError(Exception)
├─ db.py               → init_db(), insert_note(), find_notes()
└─ server.py           → add_note() and search_notes() tools, both ba
```

Step 3 — Add the Resource and the Prompt

Project: MCPForge-Local-Notes-And-Tasks-MCP-Server — Step 3 of 4: Add the Resource and the Prompt

What this step does: adds the Resource (`notes://all`) and the Prompt (`summarize_notes`) — the point isn't the code, which is small, it's deciding correctly which of the 4 things you're building belongs on which of the 3 MCP building blocks. **What's new vs. Step 2:**

`@mcp.resource("notes://all")` and `@mcp.prompt()` decorated functions in `server.py`.

What stays the same: `add_note` and `search_notes` don't change at all — they're staying Tools, on purpose, because they clearly belong there. **Why this step matters:** cramming everything into Tools "because that's the one I understand" is a real mistake real MCP servers make — a client that only wants to *read* your notes shouldn't have to go through a Tool call, with its own separate permission/approval model in most clients, just to read something. Getting this distinction right is the actual skill this project is teaching, more than the SQLite code in Step 2. **When you'll hit this for**

real: any MCP server you build past this project — the Tool/Resource/Prompt decision comes up for every single piece of functionality you add. **Read first:** [06_tools_function_calling Core Concepts — "MCP's 3 building blocks: Tools, Resources, and Prompts"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write a `list_all_notes(path) -> list[tuple]` function in `db.py` — a plain `SELECT` of every row, oldest or newest first, your choice. 2. In `server.py`, add `@mcp.resource("notes://all")` on a function that calls `list_all_notes()` and returns a formatted read-only string (one note per line, with its id and text) — no arguments, no side effects, nothing to "call" with parameters, just data to read. 3. Add `@mcp.prompt()` on a function `summarize_notes(start_date: str, end_date: str) -> str` that returns a prompt template string — not a summary itself, a *template* asking whichever model receives it to summarize notes created between those two dates. The server never calls a model here; it only hands back the text of the prompt. 4. Test both with the Inspector: read the `notes://all` resource and confirm it lists every note you've added so far (including ones from Step 2's restart test). Call `summarize_notes` with two dates and confirm you get back a well-formed prompt string, not an actual summary.

Your files after Step 3:

```
project_7_mcp_server/
├── .env / .env.example
├── config.py
├── logging_setup.py
├── exceptions.py
├── db.py
└── server.py
```

→ now also has `list_all_notes()`
→ adds `notes://all` Resource and `summarize_not`

Step 4 — A Real Client That Connects and Uses All Three

Project: MCPForge-Local-Notes-And-Tasks-MCP-Server — Step 4 of 4: A Real Client That Connects and Uses All Three

What this step does: builds a real MCP *client* — a standalone script that connects to your server, discovers what it offers, and actually exercises all three building blocks in one run. **What's new vs. Step 3:** a completely new file, `client.py` — nothing about `server.py` changes in this step. **What stays the same:** the server itself; this step proves it from the outside, the way a real client (Claude Desktop, your own agent) would use it, never touching its internals. **Why this step matters:** a server nobody has actually connected to and used isn't proven to work — the Inspector is a great manual testing tool, but a real client script proves the *exact* code path a future agent would use, works end to end, not just that a human clicking buttons in a UI got a sensible response. **When you'll hit this for real:** any time your own agent code (not a human) needs to use an MCP server's tools — this exact `stdio_client / ClientSession` shape is what a real agent's MCP integration looks like underneath. **Read first:** [06_tools_function_calling Core Concepts — "MCP servers and](#)

clients, and how they actually connect" and "Why MCP matters for the multi-agent systems this curriculum builds."

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write `client.py`: use `StdioServerParameters(command="python", args=["server.py"])` and `stdio_client(...)` to launch your server as a subprocess, then open a `ClientSession` and call `await session.initialize()`. 2. Call `await session.list_tools()`, `await session.list_resources()`, and `await session.list_prompts()` — print what each returns. This is "discovered, not hardcoded" (Doc06's phrase) in action: your client never imports anything from `server.py` directly. 3. Call `add_note` through `session.call_tool("add_note", {"text": "..."})`, then `search_notes` the same way. Read the `notes://all` resource with `session.read_resource("notes://all")`. Get the `summarize_notes` prompt with `session.get_prompt("summarize_notes", {"start_date": "...", "end_date": "..."})`. 4. Print every result. Run the whole script start to finish and confirm all 4 calls succeed in one run — this is the full loop the Charter promised, working end to end, outside of the Inspector.

Your files after Step 4 (final):

```
project_7_mcp_server/
├─ .env / .env.example
├─ config.py
├─ logging_setup.py
├─ exceptions.py
├─ db.py
├─ server.py
└─ client.py
```

(unchanged from Step 3)
→ standalone MCP client: `list_tools/list_res`

Final Deliverable: MCPForge-Local-Notes-And-Tasks-MCP-Server — a real MCP server exposing 2 Tools (`add_note` , `search_notes`), 1 Resource (`notes://all`), and 1 Prompt (`summarize_notes`), backed by a real SQLite database, plus a standalone client script that proves the full connect → discover → use loop works end to end.

Checklist Before You Call This Done

- [] Step 1's minimal server starts, and a client (Inspector or your own script) successfully calls `add_note`
- [] `add_note` and `search_notes` are backed by a real SQLite database file — notes survive a full server restart
- [] Empty or whitespace-only note text is rejected with a clear error, never silently stored
- [] `notes://all` is a Resource, not a third Tool — you can explain why
- [] `summarize_notes` is a Prompt, parameterized by a real date range, not a hardcoded string

- [] `client.py` connects, lists tools/resources/prompts, calls a tool, reads the resource, and uses the prompt — all in one run
- [] You can explain, for each of the 4 things this server exposes, why it's a Tool vs. a Resource vs. a Prompt, without help

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**